



Understanding Deadlock in Database Systems

Covers: Deadlock Occurrence · Prevention · Recovery

Join Types · SQL Examples · Best Practices

Table of Contents

01 What is Deadlock?

02 How Deadlock Occurs

03 Deadlock Prevention

04 Deadlock Recovery

05 Introduction to SQL Joins

06 Join Types & SQL Examples

What is Deadlock?

SECTION 01

A deadlock is a situation in which two or more transactions are waiting indefinitely for each other to release locks, resulting in a circular wait where none can proceed.



Circular Wait

Each transaction holds a resource and waits for another held by the next transaction.



No Progress

No transaction can proceed. All are permanently blocked, causing system stall.



System Halt

Without intervention, deadlocked transactions remain stuck indefinitely.

Real-World Analogy

Traffic Deadlock

Imagine two cars at a narrow one-lane bridge from opposite sides. Car A waits for Car B to pass, and Car B waits for Car A — neither can move!

In a Database:

- Transaction T1 locks Record A and requests a lock on Record B
- Transaction T2 locks Record B and requests a lock on Record A
- T1 waits for T2 to release B | T2 waits for T1 to release A
- **Both wait forever — DEADLOCK!**

The Four Necessary Conditions (Coffman Conditions)

1

Mutual Exclusion

A resource can be held by only one transaction at a time. Others must wait.

3

No Preemption

Resources cannot be forcibly taken from a transaction; they must be released voluntarily.

2

Hold and Wait

A transaction holds at least one resource while waiting to acquire additional resources.

4

Circular Wait

A closed chain of transactions exists where each waits for a resource held by the next.

ALL FOUR conditions must hold simultaneously for a deadlock to occur.

How Deadlock Occurs

SECTION 02

Step 1

T1 acquires lock on Resource A (e.g., Row 1 in Accounts table)

Step 2

T2 acquires lock on Resource B (e.g., Row 2 in Accounts table)

Step 3

T1 requests lock on Resource B — BLOCKED (T2 holds it)

Step 4

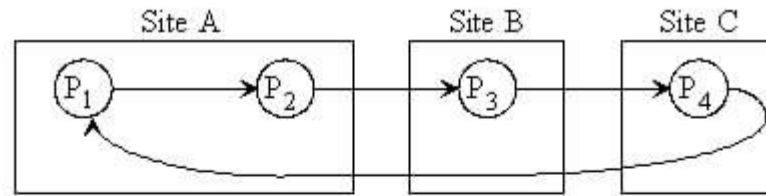
T2 requests lock on Resource A — BLOCKED (T1 holds it)

Result

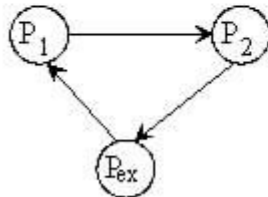
T1 and T2 wait for each other indefinitely → DEADLOCK!

Wait-For Graph (WFG)

A directed graph where nodes represent transactions and edges $T \rightarrow T'$ mean T waits for T' to release a resource.



Actual global wait-for graph contains a cycle



Local wait-for graph
at site A



Local wait-for graph
at site B



Local wait-for graph
at site C

Deadlock Example in SQL

Transaction T1

```
BEGIN TRANSACTION;

UPDATE Accounts
  SET balance = balance - 100
  WHERE id = 1;      -- Locks Row 1

-- T1 now requests Row 2
UPDATE Accounts
  SET balance = balance + 100
  WHERE id = 2;      -- BLOCKED!

COMMIT;
```

Transaction T2

```
BEGIN TRANSACTION;

UPDATE Accounts
  SET balance = balance - 200
  WHERE id = 2;      -- Locks Row 2

-- T2 now requests Row 1
UPDATE Accounts
  SET balance = balance + 200
  WHERE id = 1;      -- BLOCKED!

COMMIT;
```


Deadlock Detection

DBMS periodically checks for deadlocks using detection algorithms:



Resource Allocation Graph (RAG)

A bipartite graph tracking which transactions hold/request which resources. A cycle in RAG = deadlock for single-instance resources.



Wait-For Graph Algorithm

Simplified version of RAG for multi-instance resources. DBMS constructs this graph periodically; a cycle confirms deadlock.



Banker's Algorithm

Checks whether granting a resource request leads to an unsafe state. If unsafe, the request is deferred to prevent deadlock.

Deadlock Prevention

SECTION 03

Prevention ensures at least one of the four Coffman conditions is violated:

1

Eliminate Mutual Exclusion

Allow sharable resources. Use read-only locks where possible. Not always feasible for writes.

3

Allow Preemption

If T2 holds a resource T1 needs, rollback T2 and give its resources to T1.

2

Eliminate Hold & Wait

Require transactions to request ALL resources at once before execution begins.

4

Eliminate Circular Wait

Impose a total ordering on resource types. Transactions must request in increasing order.

Timestamp-Based Prevention Schemes

Use transaction timestamps (older = higher priority) to break hold-and-wait:

Wait-Die Scheme

Older Txn requests resource held by Younger Txn:

→ Older **WAITS** (non-preemptive)

Younger Txn requests resource held by Older Txn:

→ Younger **DIES** (rolls back & restarts)

Rule: Old waits, Young dies

Wound-Wait Scheme

Older Txn requests resource held by Younger Txn:

→ Older **WOUNDS** Younger (preempts it)

Younger Txn requests resource held by Older Txn:

→ Younger **WAITS**

Rule: Old wounds, Young waits

Lock Ordering & Two-Phase Locking (2PL)

Lock Ordering Strategy

Assign each resource a unique number. Transactions must always request resources in **strictly increasing order** of their resource numbers. This eliminates circular wait.

Example: Always lock Resource #1 before Resource #2. Never request #1 after already requesting #2.

Two-Phase Locking (2PL)

Growing Phase: Transaction acquires all needed locks. No locks are released during this phase.

Shrinking Phase: Transaction releases locks. No new locks can be acquired in this phase.

2PL guarantees serializability but does NOT fully prevent deadlocks without strict 2PL variant.

Deadlock Recovery

SECTION 04

Once a deadlock is detected, the DBMS must break it by aborting one or more transactions:



Transaction Abortion

Select a victim transaction and abort it. Its locks are released, allowing other transactions to proceed. The aborted transaction must be restarted.



Rollback

Instead of full abort, roll back a transaction to a savepoint or checkpoint. Partial rollback reduces overhead compared to complete restart.



Preemption (Resource Seizure)

Forcibly take a resource from a transaction and give it to another. The preempted transaction is suspended or rolled back until the resource is freed.

Victim Selection Criteria

When aborting a transaction to resolve a deadlock, the DBMS chooses a victim based on cost:

Minimum Cost:

Abort the transaction that has done the least work (fewest updates, lowest rollback cost).

Fewest Resources Held:

Abort the transaction holding the fewest locks to minimize the disruption to other waiting transactions.

Most Resources Needed:

If a transaction needs many more resources, aborting it unblocks more transactions at once.

Starvation Avoidance:

Track how many times a transaction has been rolled back. Prefer different victims to avoid repeated starvation of one transaction.

Transaction Age:

Younger transactions (with smaller timestamps) are preferred as victims since they've done less work.

Rollback Strategies in Recovery

Total Rollback

Abort the victim transaction completely and restart from scratch. Simple but expensive for long-running transactions.

Partial Rollback

Roll back only as far as necessary to break the deadlock (to a savepoint). Less overhead, but requires savepoint support.

Prevention vs. Detection & Recovery

Aspect	Prevention	Detection & Recovery
Approach	Avoid conditions for deadlock	Allow deadlocks, detect & fix
Overhead	High (extra checks per request)	Lower continuous overhead
Throughput	May reduce concurrency	Higher concurrency
Complexity	Moderate	Higher (detection algorithm needed)
Resource Use	May under-utilize resources	Better utilization
Best For	Short, predictable transactions	Long, unpredictable transactions

Deadlock — Key Takeaways



Deadlock = circular wait where no transaction can proceed



4 Coffman Conditions must ALL hold: Mutex, Hold&Wait, No Preemption, Circular Wait



Detection uses WFG, RAG, or Banker's Algorithm to find cycles



Prevention breaks one condition: ordering, timestamps, or pre-claiming resources



Recovery aborts or rolls back victim transactions to break the cycle



Real DBMSs (MySQL, PostgreSQL, Oracle) auto-detect & resolve deadlocks



Introduction to

SQL JOINS

Combining data from multiple tables efficiently

SECTION 05

What is a JOIN?

A JOIN clause is used to combine rows from two or more tables based on a related column (usually a foreign key). JOINS are the foundation of relational queries.

Example Tables:

Employees		
emp_id	name	dept_id
1	Alice	10
2	Bob	20
3	Carol	10
4	Dave	30
5	Eve	50

Common key: dept_id

Departments	
dept_id	dept_name
10	Engineering
20	Marketing
30	HR
40	Finance

Types of SQL JOINS

INNER JOIN

Returns only matching rows from both tables.

LEFT (OUTER) JOIN

All rows from left table + matched rows from right.

RIGHT (OUTER) JOIN

All rows from right table + matched rows from left.

FULL OUTER JOIN

All rows from both tables, matched or unmatched.

CROSS JOIN

Cartesian product — every row combined with every other row.

SELF JOIN

A table joined with itself to compare rows.

INNER JOIN

SECTION 06

Returns rows where there is a match in BOTH tables.

```
SELECT e.emp_id, e.name, d.dept_name
FROM Employees e
INNER JOIN Departments d ON e.dept_id = d.dept_id;
```

emp_id	name	dept_name
1	Alice	Engineering
2	Bob	Marketing
3	Carol	Engineering
4	Dave	HR

*Dave (dept_id=30)
would be excluded if
dept_id 30 didn't exist
in Departments.*

LEFT (OUTER) JOIN

Returns ALL rows from the left table and matching rows from the right. NULLs fill unmatched right rows.

```
SELECT e.emp_id, e.name, d.dept_name
FROM Employees e
LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

emp_id	name	dept_name
1	Alice	Engineering
2	Bob	Marketing
3	Carol	Engineering
4	Dave	HR
5	Eve	NULL ← no matching dept

RIGHT (OUTER) JOIN

Returns ALL rows from the right table and matching rows from the left. NULLs fill unmatched left rows.

```
SELECT e.emp_id, e.name, d.dept_name
FROM Employees e
RIGHT JOIN Departments d ON e.dept_id = d.dept_id;
```

emp_id	name	dept_name
1	Alice	Engineering
2	Bob	Marketing
3	Carol	Engineering
4	Dave	HR
NULL	NULL	Finance ← no employee

FULL OUTER JOIN

Returns ALL rows from both tables. Unmatched rows get NULL for the other table's columns.

```
SELECT e.emp_id, e.name, d.dept_name
FROM Employees e
FULL OUTER JOIN Departments d ON e.dept_id = d.dept_id;
-- Note: MySQL uses UNION of LEFT and RIGHT JOIN instead
```

Result includes:
matched rows +
unmatched from Employees +
unmatched from Departments:

emp_id	name	dept_name
1	Alice	Engineering
3	Carol	Engineering
2	Bob	Marketing
4	Dave	HR
5	Eve	NULL (no dept)
NULL	NULL	Finance (no emp)

CROSS JOIN & SELF JOIN

CROSS JOIN — Cartesian Product

Returns every combination of rows. If Table A has 4 rows and Table B has 3 rows, result has 12 rows.

```
SELECT e.name, d.dept_name
FROM Employees e CROSS JOIN Departments d;
-- Result: 4 employees × 3 departments = 12 rows
```

SELF JOIN — Join a Table with Itself

Used to compare rows within the same table (e.g., find employees and their managers).

```
SELECT e.name AS Employee, m.name AS Manager
FROM Employees e JOIN Employees m ON e.manager_id = m.emp_id;
```

JOIN with Multiple Conditions & Aliases

JOIN with WHERE Filter

```
SELECT e.name, d.dept_name, e.salary
FROM Employees e
INNER JOIN Departments d ON e.dept_id = d.dept_id
WHERE d.dept_name = 'Engineering'
      AND e.salary > 50000;
```

JOIN with ORDER BY & LIMIT

```
SELECT e.name, d.dept_name, e.salary
FROM Employees e
LEFT JOIN Departments d ON e.dept_id = d.dept_id
ORDER BY e.salary DESC
LIMIT 5;
```

Joining Three or More Tables

Extend JOINS to link multiple tables in one query:

```
-- Schema: Employees → Departments → Locations
SELECT e.name, d.dept_name, l.city
FROM Employees e
INNER JOIN Departments d ON e.dept_id = d.dept_id
INNER JOIN Locations l    ON d.location_id = l.location_id
WHERE l.country = 'Bangladesh'
ORDER BY e.name;
```

Explanation:

1. Employees JOIN Departments on dept_id → gets department for each employee
2. JOIN Locations on location_id → gets city for each department
3. WHERE filters to employees in Bangladesh only

JOINS with GROUP BY & Aggregates

Count Employees per Department

```
SELECT d.dept_name, COUNT(e.emp_id) AS total_employees
FROM Departments d
LEFT JOIN Employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name
ORDER BY total_employees DESC;
```

Average Salary per Department

```
SELECT d.dept_name,
       AVG(e.salary) AS avg_salary,
       MAX(e.salary) AS max_salary,
       MIN(e.salary) AS min_salary
FROM Departments d
INNER JOIN Employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name
HAVING AVG(e.salary) > 60000;
```

Subquery vs JOIN & Performance Tips

Subquery Approach

```
SELECT name FROM Employees
WHERE dept_id IN (
    SELECT dept_id
    FROM Departments
    WHERE dept_name = 'HR'
);
```

JOIN Approach (Preferred)

```
SELECT e.name
FROM Employees e
INNER JOIN Departments d
    ON e.dept_id = d.dept_id
WHERE d.dept_name = 'HR';
```

Performance Tips:

- ✓ Index the JOIN columns (foreign keys) for faster lookups
- ✓ Prefer INNER JOIN over OUTER JOIN when NULLs aren't needed
- ✓ Use SELECT only needed columns; avoid SELECT *
- ✓ Use EXPLAIN / EXPLAIN ANALYZE to inspect query execution plans